Contents

Midterm Practical Lab Preparation Packet 1
Instructions & Ground Rules 1
Study Topic Checklist 1
Shared SQL Database Setup (For Questions 15–18 & 20–22) 2
Part 1: Warm-Up Tasks (Questions 1–6) 3
Question 1: Order Invoice Calculator & Type Inspector (5 pts) 3
Question 2: Student Gradebook Array Processor (5 pts) 4
Question 3: Shipping Rate Decision Engine (`if`/`elseif`/`else` and `switch`) (5 pts) 4
Question 4: HTTP Status Dispatcher (match Expression) (5 pts) 5
Question 5: Loops Showdown (`for`, `while`, `do...while`) (5 pts) 6
Question 6: Modular Page Architecture (require_once and include) (5 pts) 6
Part 2: Core Tasks — Functions & Browser Data (Questions 7–14) 7
Question 7: Custom Pricing Functions, Default Values, and PHPDoc (5 pts) 7
Question 8: Strict Types, Nullable Types & Named Arguments (5 pts) 8
Question 9: Variable Scope (static vs. local) & Built-In Functions (5 pts) 9
Question 10: Query Strings, $_GET, Missing Keys & Allowed-List Validation (5 pts) 9
Question 11: HTML Forms, $_POST, Text Length Validation & Sticky Inputs (5 pts) 10
Question 12: Number Ranges & PHP Filter Functions (filter_var) (5 pts) 11
Question 13: Password Rules & Checkbox Array Validation (5 pts) 11
Question 14: XSS Defense & Redirect-After-Success Pattern (5 pts) 12
Part 3: Core Tasks — Introductory SQL (Questions 15–18) 13
Question 15: SQL Table Design, Keys, and Data Types (q15_books_schema.sql) (5 pts) 13
Question 16: Basic SQL SELECT, WHERE, and ORDER BY (q16_catalog_queries.sql) (5 pts) 14
Question 17: SQL Pagination Queries with LIMIT and OFFSET (q17_pagination_queries.sql) (5 pts) 14
Question 18: SQL Aggregate Functions (q18_aggregate_queries.sql) (5 pts) 15
Part 4: Challenge & Integrated Mini-App Tasks (Questions 19–22) 16
Question 20 (Integrated Mini-App 1): University Course Finder & Tuition Estimator (10 pts) 17
Question 21 (Integrated Mini-App 2): Hardware B2B Quote Builder with Redirect-After-Success (10 pts) 18
Question 22 (Integrated Mini-App 3): Storefront Explorer & Quick-Order Redirect App (10 pts) 19

Midterm Practical Lab Preparation Packet

- **Course Focus**: Core PHP 8+ & Introductory SQL
- **Total Questions**: 22 Hands-On Lab Tasks (130 Total Points)
- **Target Environment**: Local XAMPP Server (Apache + MySQL + PHP 8.1+)

Instructions & Ground Rules

1. **Workspace Setup:** Create a folder named midterm_lab inside your XAMPP htdocs directory (C:\xampp\htdocs\midterm_lab\ on Windows or /Applications/XAMPP/htdocs/midterm_lab/ on macOS).
2. **Verifying Your Work:**
   - **PHP Tasks:** Start **Apache** in the XAMPP Control Panel and test every .php script in your browser at http://localhost/midterm_lab/<filename>.php.
   - **Introductory SQL Tasks:** Start **MySQL** in the XAMPP Control Panel, open **phpMyAdmin** at http://localhost/phpmyadmin, and execute your .sql queries in the **SQL** tab to verify the exact output rows and columns.
3. **Strict Course Scope Constraints:**
   - **Allowed:** Core PHP 8+, built-in PHP functions, HTML forms ($_GET, $_POST), htmlspecialchars(), filter_var() / filter_input(), modular file inclusion (require_once, include), and introductory SQL (CREATE TABLE, SELECT, WHERE, ORDER BY, LIMIT/OFFSET, and basic aggregate functions).
   - **Not Allowed (Not Covered):** No frameworks, no Composer packages, no database driver/PDO-specific code patterns in PHP, and no sessions ($_SESSION) or cookies.
   - **Redirect-After-Success Pattern:** Whenever a task requires redirecting after a valid form submission, pass the confirmed data via a URL query string using urlencode() or http_build_query(), send the redirect header with header('Location: ...'), and terminate the script immediately with exit;.
   - **XSS Escaping & Error-Free Execution:** Escape every dynamic value output into HTML with htmlspecialchars((string)$value, ENT_QUOTES, 'UTF-8'). Pages must load with zero PHP warnings or notices even before a form is submitted.

Study Topic Checklist

Topic Area	Specific Skills Tested	Questions
PHP Fundamentals	Variables, scalar data types, loose (==) vs. strict (===) comparison, arithmetic/string/logical operators, indexed/associative/multidimensional arrays	Q1, Q2, Q19
Control Structures	if/elseif/else, switch, PHP 8 match, for, while, do...while, foreach, include/require/_once	Q2, Q3, Q4, Q5, Q6, Q19, Q21
Functions	Defining/calling functions, parameters/arguments, return values, local/global/static scope, declare(strict_types=1), type hints, nullable types, default values, named arguments, PHPDoc, functions.php, built-in functions	Q7, Q8, Q9, Q19, Q20, Q21, Q22
Browser Data & Security	$_GET, $_POST, missing keys (isset, ??), allowed lists (in_array), number ranges, text length, passwords, checkboxes, run/redisplay/redirect (header('Location: ...') + exit), htmlspecialchars(), filter_var() / filter_input()	Q10, Q11, Q12, Q13, Q14, Q19, Q20, Q21, Q22

Introductory SQL	Tables, primary keys, SQL data types, project setup in phpMyAdmin, SELECT, WHERE, ORDER BY, LIMIT/OFFSET, aggregate functions (COUNT, AVG, MIN, MAX, SUM)	Q15, Q16, Q17, Q18, Q20, Q21, Q22
Integrated Mini-Apps	End-to-end PHP + sticky HTML forms + validation + escaping + functions + arrays/loops + redirect-after-success + companion SQL SELECT queries	Q20, Q21, Q22

Shared SQL Database Setup (For Questions 15–18 & 20–22)

Before starting the SQL questions, open **phpMyAdmin** (http://localhost/phpmyadmin), click the **SQL** tab, paste the script below, and click **Go** to create the webprog2_midterm database and sample tables.

```
CREATE DATABASE IF NOT EXISTS webprog2_midterm
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE webprog2_midterm;

DROP TABLE IF EXISTS products;
CREATE TABLE products (
    product_id INT AUTO_INCREMENT PRIMARY KEY,
    sku VARCHAR(20) NOT NULL,
    product_name VARCHAR(100) NOT NULL,
    category VARCHAR(40) NOT NULL,
    unit_price DECIMAL(8, 2) NOT NULL,
    stock_qty INT NOT NULL,
    is_featured TINYINT(1) NOT NULL DEFAULT 0,
    added_date DATE NOT NULL
);

INSERT INTO products (sku, product_name, category, unit_price, stock_qty,
is_featured, added_date) VALUES
('HW-101', 'Mechanical Keyboard RGB', 'Peripherals', 89.99, 24, 1, '2026-01-
15'),
('HW-102', 'Wireless Ergonomic Mouse', 'Peripherals', 45.50, 50, 0, '2026-
01-20'),
('HW-103', '27-inch 4K IPS Monitor', 'Monitors', 329.00, 12, 1, '2026-02-
01'),
('HW-104', 'USB-C 8-in-1 Hub', 'Accessories', 34.95, 85, 0, '2026-02-10'),
('HW-105', '1TB NVMe M.2 SSD', 'Storage', 79.99, 30, 1, '2026-02-14'),
('HW-106', '2TB Portable External SSD', 'Storage', 139.50, 18, 0, '2026-02-
18'),
('HW-107', '34-inch Ultrawide Monitor', 'Monitors', 499.99, 5, 1, '2026-02-
22'),
('HW-108', 'Noise-Canceling USB Headset', 'Audio', 64.00, 40, 0, '2026-03-
01'),
```

```
('HW-109', 'Braided HDMI 2.1 Cable 2m', 'Accessories', 14.99, 120, 0, '2026-
03-05'),
('HW-110', 'Studio Condenser Microphone', 'Audio', 115.00, 9, 1, '2026-03-
10'),
('HW-111', 'Laptop Aluminum Stand', 'Accessories', 29.99, 60, 0, '2026-03-
12'),
('HW-112', '4TB Desktop HDD 7200RPM', 'Storage', 94.00, 15, 0, '2026-03-
15');

DROP TABLE IF EXISTS courses;
CREATE TABLE courses (
    course_id INT AUTO_INCREMENT PRIMARY KEY,
    course_code VARCHAR(15) NOT NULL,
    title VARCHAR(100) NOT NULL,
    department VARCHAR(30) NOT NULL,
    credits INT NOT NULL,
    lab_fee DECIMAL(6, 2) NOT NULL,
    max_seats INT NOT NULL,
    is_online TINYINT(1) NOT NULL DEFAULT 0
);

INSERT INTO courses (course_code, title, department, credits, lab_fee,
max_seats, is_online) VALUES
('CS101', 'Introduction to Programming', 'CS', 4, 50.00, 35, 0),
('CS202', 'Web Programming 2 (PHP & SQL)', 'CS', 4, 75.00, 30, 0),
('CS215', 'Database Management Systems', 'CS', 3, 60.00, 30, 1),
('IT110', 'Networking Fundamentals', 'IT', 3, 85.00, 25, 0),
('IT240', 'Cloud Infrastructure Basics', 'IT', 3, 40.00, 40, 1),
('DS105', 'UI/UX Design Principles', 'Design', 3, 30.00, 28, 1),
('DS220', 'Responsive Frontend Systems', 'Design', 4, 55.00, 24, 0),
('CS301', 'Software Engineering Lab', 'CS', 4, 90.00, 20, 0);
```

Part 1: Warm-Up Tasks (Questions 1–6)

Question 1: Order Invoice Calculator & Type Inspector (5 pts)

- **Learning Objective**: Declare variables of different scalar data types, apply arithmetic and string operators, and distinguish between loose (==) and strict (===) comparison operators.
- Scenario: An e-commerce checkout diagnostic page needs to compute an order total from raw product data and verify why a string promo code ("100") and an integer promo code (100) behave differently under loose and strict comparisons.
- Starter File (q01_invoice_calc.php)
- **Requirements:**
    - Calculate $subtotal ($unitPrice * $quantity), $taxAmount ($subtotal * $taxRate), and $grandTotal ($subtotal + $taxAmount).

    - Build a single summary string $receiptHeader using the concatenation operator (.) or string interpolation that reads: "Customer: Alex Rivera | Item: Mechanical Keyboard".
    - Store the boolean result of $promoInput == $targetPromo in $looseMatch and $promoInput === $targetPromo in $strictMatch.
    - Display the variables, their PHP data types using gettype(), the formatted dollar amounts (to 2 decimal places using number_format()), and the boolean comparison results (true or false via var_export($val, true)).
- **Expected Output / Behavior:**
    - Displays the concatenated header string.
    - Shows Subtotal: $269.97, Tax (8%): $21.60, and Grand Total: $291.57.
    - Shows Loose Match (==): true and Strict Match (===): false alongside the data types (string vs. integer).
- **Constraints:** Do not hardcode calculated numbers; use PHP arithmetic operators.
- **Self-Check & Grading Criteria:**
    - [ ] Correct arithmetic formulas for subtotal, tax, and grand total (2 pts)
    - [ ] Proper string concatenation or interpolation (1 pt)
    - [ ] Accurate demonstration of == vs. === and gettype() (2 pts)

Question 2: Student Gradebook Array Processor (5 pts)

- **Learning Objective:** Create and iterate over associative arrays using foreach loops to compute summary statistics and render an HTML table.
- **Scenario:** A course assistant needs a script that displays student exam scores from an associative array, flags whether each student passed (score of 70 or higher), and calculates the class average.
- **Starter File (q02_gradebook.php)**
- **Requirements:**
    1. Iterate over $scores using foreach ($scores as $student => $score).
    2. Render an HTML table with columns: **Student Name**, **Score**, and **Status** ("Pass" if score is 70 or higher, otherwise "Retake").
    3. During iteration, accumulate the total score and count how many students passed.
    4. Below the table, display the total number of students, the number of passing students, and the class average formatted to 1 decimal place.
- **Expected Output / Behavior:** An HTML table listing all 6 students (Park Seo-yeon and Jung Woo-jin marked as Retake, the other 4 marked as Pass), followed by Total Students: 6 | Passed: 4 | Class Average: 78.8.
- **Constraints:** Must use foreach to compute the sum and pass count (do not hardcode the summary numbers).
- **Self-Check & Grading Criteria:**
    - [ ] Key-value foreach syntax used accurately (2 pts)
    - [ ] Pass/Retake status determined per student (1 pt)
    - [ ] Accurate average calculation and HTML table output (2 pts)

Question 3: Shipping Rate Decision Engine (`if`/`elseif`/`else` and `switch`) (5 pts)

- **Learning Objective:** Implement multi-branch selection logic using `if`/`elseif`/`else` for numeric range checks and `switch` for discrete category codes.

- **Scenario:** A warehouse calculator determines the base shipping cost based on package weight (in kg) and adds a zone surcharge based on a shipping region code (`"LOCAL"`, `"REGIONAL"`, `"INTL"`).
- **Starter File (`q03_shipping_tier.php`)**
- **Requirements:**
  1. Use an if / elseif / else structure to determine $baseCost:
     - Weight less than or equal to 0: invalid weight ($baseCost = 0.0, set $error = "Invalid package weight.").
     - Weight up to 2.0 kg: $5.00.
     - Weight above 2.0 kg up to 10.0 kg: $12.50.
     - Weight above 10.0 kg: $25.00.
  2. Use a switch ($zoneCode) statement to determine $surcharge:
     - 'LOCAL': $0.00
     - 'REGIONAL': $4.50
     - 'INTL': $18.00
     - default: $10.00 (unclassified zone)
  3. Display the weight, zone code, base cost, zone surcharge, and total shipping cost (or the error message if weight is invalid).
- **Expected Output / Behavior:** For 6.5 kg and 'REGIONAL', outputs Base Cost: $12.50, Zone Surcharge: $4.50, Total Shipping: $17.00. Test your script by changing $weightKg and $zoneCode to verify all branches.
- **Constraints:** Must use if/elseif/else for the weight brackets and switch (with break; statements) for the zone code.
- **Self-Check & Grading Criteria:**

  - [ ] Proper boundary conditions in if/elseif/else (2 pts)
  - [ ] Valid switch structure including break and default (2 pts)
  - [ ] Correct total output (1 pt)

Question 4: HTTP Status Dispatcher (match Expression) (5 pts)

- **Learning Objective:** Use PHP 8+ match expressions with multiple arms, default fallback, and strict identity comparison.
- **Scenario:** A system status widget maps integer HTTP status codes to human-readable status messages without falling prey to switch's loose type coercion.
- **Starter File (q04_http_status_matcher.php)**

- **Requirements:**
  1. Loop through $statusCodes.
  2. Inside the loop, assign $description using a single PHP 8 match ($code) expression:
     1. 200, 201 maps to 'Success: Request completed'
     2. 301, 302 maps to 'Redirect: Resource moved'
     3. 400, 401, 403 maps to 'Client Error: Bad request or unauthorized'
     4. 404 maps to 'Client Error: Not Found'
     5. 500, 503 maps to 'Server Error: Internal failure'
     6. default maps to 'Unknown or non-integer status code'
  3. Output each code, its data type (gettype($code)), and the matched description in an HTML table.

- **Expected Output / Behavior:** Integer 200 and 201 output 'Success: Request completed', whereas string '200' hits the default arm ('Unknown or non-integer status code') because match performs strict (===) comparison.
- **Constraints:** Must use match (not switch or if).
- **Self-Check & Grading Criteria:**
  - [ ] Correct match syntax returning a value directly to a variable (2 pts)
  - [ ] Comma-separated multi-value match arms used (2 pts)
  - [ ] Default arm catches string '200' and 999 (1 pt)

Question 5: Loops Showdown (`for`, `while`, `do...while`) (5 pts)

- **Learning Objective:** Apply `for`, `while`, and `do...while` loops appropriately to solve counter-controlled, condition-controlled, and post-test iteration problems.
- **Scenario:** A financial utility script needs three sections: (1) a 6-month savings projection (`for`), (2) a loan payoff simulator that deducts monthly payments until the balance reaches zero (`while`), and (3) a retry connection tick that always runs at least once (`do...while`).
- **Starter File (`q05_loan_amortization.php`):**

- **Requirements:**
  1. **for loop:** Run from month 1 to 6, adding $monthlyDeposit each month to $initialSavings and printing each month's balance.
  2. **while loop:** As long as $loanBalance is greater than 0, increment a payment counter, subtract $monthlyPayment (if remaining balance is less than $monthlyPayment, set the final payment equal to the remaining balance and balance to 0), and output each payment step.
  3. **do...while loop:** Output "Diagnostic ping attempt #$attempt", increment $attempt, and continue while $attempt is less than or equal to $maxAttempts.
- **Expected Output / Behavior:**
  - Section 1 shows Months 1 to 6 ending at $1,400.00.
  - Section 2 shows 4 monthly payments ($300.00, $300.00, $300.00, $100.00) reaching $0.00 balance in 4 months.
  - Section 3 outputs "Diagnostic ping attempt #1" once.
- **Constraints:** Must use one for, one while, and one do...while loop.
- **Self-Check & Grading Criteria:**
  - [ ] for loop accurately tracks 6 months (2 pts)
  - [ ] while loop terminates cleanly without infinite looping or negative final balance (2 pts)
  - [ ] do...while executes at least once (1 pt)

Question 6: Modular Page Architecture (require_once and include) (5 pts)

- **Learning Objective:** Organize a multi-file PHP application using require_once and include, preventing duplicate constant or function redeclarations.
- **Scenario:** A company portal needs a shared configuration file, a reusable header template, a main page, and a footer template so that site-wide constants and helper functions are loaded once and never duplicated.
- **Starter Files:** Create a subfolder q06_layout_includes/ containing four files: config.php, header.php, footer.php, and index.php.

- **Requirements:**

1. In config.php, define two constants: SITE_NAME ("Busan Tech Portal") and SUPPORT_EMAIL ("help@busantech.kr"), plus a function getPortalVersion(): string returning "v2.4".
2. In header.php, use require_once __DIR__ . '/config.php'; and render the opening HTML boilerplate and a main heading displaying SITE_NAME.
3. In footer.php, render a footer section showing SUPPORT_EMAIL and getPortalVersion(), and close the HTML document tags.
4. In index.php, call require_once __DIR__ . '/config.php';, then include __DIR__ . '/header.php';, output main page content, and finally include __DIR__ . '/footer.php';.

- **Expected Output / Behavior:** Loading q06_layout_includes/index.php renders the complete page without "Constant SITE_NAME already defined" or "Cannot redeclare getPortalVersion()" errors, even though both index.php and header.php load config.php.
- **Constraints:** Must use require_once for config.php so duplicate inclusions do not cause a fatal redeclaration error.
- **Self-Check & Grading Criteria:**
  - [ ] require_once used for config.php in both index.php and header.php (2 pts)
  - [ ] __DIR__ path resolution used properly (1 pt)
  - [ ] Constants and function render without redeclaration errors (2 pts)

Part 2: Core Tasks — Functions & Browser Data (Questions 7–14)

Question 7: Custom Pricing Functions, Default Values, and PHPDoc (5 pts)

- **Learning Objective:** Define and call custom functions with default parameter values, return values, and complete PHPDoc documentation blocks.
- **Scenario:** A retail checkout script needs reusable functions to calculate discounted prices and format currency strings with a configurable currency symbol.
- **Starter File:** Create q07_pricing_functions.php.
- **Requirements:**
  1. Write a PHPDoc block (/** ... */ with @param and @return tags) and define:

```
function calculateNetPrice(float $basePrice, float
$discountPercent = 0.0, float $taxPercent = 10.0): float
```

     - Formula: subtract $basePrice * ($discountPercent / 100), then add $taxPercent% tax on the discounted price.
  2. Write a PHPDoc block and define:

```
function formatCurrency(float $amount, string $symbol = '$'):
string
```

     - Formula: subtract $basePrice * ($discountPercent / 100), then add $taxPercent% tax on the discounted price.
  3. Call calculateNetPrice() three times in your script:
     - Call 1: Pass only $basePrice (100.00) — relying on both default parameters.
     - Call 2: Pass $basePrice (100.00) and $discountPercent (20.0) — relying on default tax.
     - Call 3: Pass all three arguments (100.00, 20.0, 5.0).
  4. Format and output all three results using formatCurrency().
- **Expected Output / Behavior:**

- Call 1 ($100, 0% off, 10% tax): $110.00
- Call 2 ($100, 20% off, 10% tax): $88.00
- Call 3 ($100, 20% off, 5% tax): $84.00

- **Constraints:** Functions must return values (never `echo` directly inside the functions).
- **Self-Check & Grading Criteria:**
  - [ ] Complete PHPDoc comments with `@param` and `@return` (1 pt)
  - [ ] Correct default parameter syntax and math (2 pts)
  - [ ] Functions return values rather than echoing directly (2 pts)

Question 8: Strict Types, Nullable Types & Named Arguments (5 pts)

- **Learning Objective:** Enforce strict type checking with declare(strict_types=1);, use parameter and return type declarations (including nullable ?string), and invoke functions with PHP 8 named arguments.
- **Scenario:** A shipping label generator stores its helper functions in a separate q08_functions.php file and requires strict typing so accidental string-to-number coercion is caught immediately.
- **Starter Files:** Create q08_functions.php and q08_strict_shipping.php.
- **Requirements:**
  1. Place declare(strict_types=1); at the very top of both q08_functions.php and q08_strict_shipping.php.
  2. In q08_functions.php, define:

```php
function buildShippingLabel(
    string $recipient,
    float $weightKg,
    bool $fragile = false,
    bool $express = false,
    ?string $deliveryNote = null
): string
```

- Return a formatted summary string including all details (if $deliveryNote is null, display "None").
  3. In q08_strict_shipping.php, load q08_functions.php using require_once.
  4. Call buildShippingLabel() twice using **PHP 8 named arguments**:
     - Call 1: Pass recipient: 'Kim Tae-hyung', weightKg: 4.5, skip $fragile, and set express: true and deliveryNote: 'Leave at front desk'.
     - Call 2: Pass arguments in a custom order (weightKg: 1.25, recipient: 'Choi Ji-woo', fragile: true).

- **Expected Output / Behavior:** Both labels display accurately with their respective flags (Express: YES on Call 1; Fragile: YES and Note: None on Call 2).
- **Constraints:** declare(strict_types=1); must be the first statement in both files. Named arguments must be used in q08_strict_shipping.php.
- **Self-Check & Grading Criteria:**
  - [ ] declare(strict_types=1); placed at top of both files (1 pt)
  - [ ] Correct nullable type ?string and return type : string (2 pts)
  - [ ] Proper use of PHP 8 named arguments skipping middle default parameters (2 pts)

Question 9: Variable Scope (static vs. local) & Built-In Functions (5 pts)

- **Learning Objective:** Demonstrate local and static variable scope inside a function while combining built-in PHP string and array functions (trim, strtoupper, substr, implode, str_pad, ucwords).
- **Scenario:** An inventory intake utility generates sequential SKU codes (WH-PER-001, WH-MON-002, etc.) from messy product category strings during a single page execution without using global variables.
- **Starter File (q09_sku_generator.php)**
- **Requirements**:
    1. Define generateNextSku(string $category, string $prefix = 'WH'): string.
    2. Inside generateNextSku(), declare a static $sequence = 0; variable and increment it on every call.
    3. Clean $category using trim(), convert it to uppercase using strtoupper(), and extract the first 3 characters using substr().
    4. Pad $sequence to 3 digits with leading zeros using str_pad((string)$sequence, 3, '0', STR_PAD_LEFT).
    5. Join $prefix, the 3-letter category code, and the padded sequence number with hyphens using implode('-', [...]) and return it.
    6. Loop through $rawItems, clean each item name with ucwords(trim($item['name'])), call generateNextSku(), and render the results in an HTML list.
- **Expected Output / Behavior**: Outputs 4 items with SKUs WH-PER-001, WH-MON-002, WH-ACC-003, and WH-STO-004 alongside cleaned title-case product names.
- **Constraints**: Do not use global variables to track the sequence counter; must use a static local variable inside generateNextSku().
- **Self-Check & Grading Criteria:**
    - [ ] static variable retains its state across function calls (2 pts)
    - [ ] Proper use of trim, strtoupper, substr, str_pad, and implode (2 pts)
    - [ ] No reliance on global keyword (1 pt)

Question 10: Query Strings, $_GET, Missing Keys & Allowed-List Validation (5 pts)

- **Learning Objective:** Safely read query parameters from $_GET, handle missing keys with the null-coalescing operator (??), validate against an allowed list (in_array with strict mode), and escape output.
- **Scenario:** A documentation reader page lets users switch the display theme (light, dark, sepia) and font size (small, medium, large) via URL query parameters (?theme=dark&size=large). Tampered URL values (such as ?theme=neon&size=giant) must fall back to safe defaults and display a warning notice.
- **Starter File:** Create q10_theme_switcher.php.
- **Requirements:**

    1. Define allowed lists:

        - $allowedThemes = ['light', 'dark', 'sepia'];
        - $allowedSizes = ['small', 'medium', 'large'];

    2. Read $_GET['theme'] ?? 'light' and $_GET['size'] ?? 'medium'.

3. Validate both values using in_array($value, $allowedList, true). If an invalid value is passed, reset it to the default ('light' or 'medium') and store a warning message in a $warnings array.
4. Render clickable test links on the page:
    - ?theme=light&size=medium
    - ?theme=dark&size=large
    - ?theme=sepia&size=small
    - ?theme=neon&size=giant (invalid test link)
5. Apply visual styling or display the active validated settings and any warnings safely using htmlspecialchars().

- **Expected Output / Behavior:** Clicking valid links changes the active theme and size; clicking ?theme=neon&size=giant displays the warning messages and falls back to light and medium without throwing any Undefined array key warnings.
- **Constraints:** Must use ?? (or isset()) and in_array(..., ..., true).
- **Self-Check & Grading Criteria:**
    - [ ] Zero PHP warnings when visiting q10_theme_switcher.php with no query string (2 pts)
    - [ ] Strict allowed-list check via in_array($val, $list, true) (2 pts)
    - [ ] All output properly escaped with htmlspecialchars() (1 pt)

Question 11: HTML Forms, $_POST, Text Length Validation & Sticky Inputs (5 pts)

- **Learning Objective:** Process an HTML POST form, validate text length boundaries (mb_strlen or strlen), redisplay the form with sticky user inputs on error, and show a success confirmation when valid.
- **Scenario:** A course feedback form requires a student's name (between 2 and 50 characters) and a feedback comment (between 10 and 300 characters). If validation fails, the student should not lose what they typed.
- **Starter File:** Create q11_feedback_form.php.
- **Requirements:**
    1. Check $_SERVER['REQUEST_METHOD'] === 'POST'.
    2. Read and trim() $_POST['student_name'] ?? '' and $_POST['comment'] ?? ''.
    3. Validate:
        - $studentName: character length must be between 2 and 50 inclusive.
        - $comment: character length must be between 10 and 300 inclusive.
    4. If errors exist, display an error list above the form and repopulate both fields (value="..." for the text input and inner text for the textarea element) using htmlspecialchars().
    5. If valid, hide the form and display a confirmation card showing the escaped input values.
- **Expected Output / Behavior:** Submitting "A" and "Too short" shows validation errors for both fields while keeping "A" and "Too short" inside the form controls. Submitting valid lengths renders the confirmation message.

- **Constraints:** Must use method="post", trim whitespace before checking length, and escape sticky values.
- **Self-Check & Grading Criteria:**
  - [ ] Checks $_SERVER['REQUEST_METHOD'] === 'POST' (1 pt)
  - [ ] Trims inputs and checks min/max character lengths accurately (2 pts)
  - [ ] Sticky input and textarea values properly escaped with htmlspecialchars() (2 pts)

Question 12: Number Ranges & PHP Filter Functions (filter_var) (5 pts)

- **Learning Objective:** Validate integer ranges, floats, and email addresses using PHP's built-in filter functions (filter_var, FILTER_VALIDATE_INT, FILTER_VALIDATE_FLOAT, FILTER_VALIDATE_EMAIL).
- **Scenario:** A conference ticket reservation form collects an attendee's email, number of tickets (integer between 1 and 5), and optional donation amount (float between 0.00 and 500.00).
- **Starter File:** Create q12_ticket_order.php.
- **Requirements:**
  1. Accept POST submissions with fields email, tickets, and donation.
  2. Validate email using filter_var($rawEmail, FILTER_VALIDATE_EMAIL).
  3. Validate tickets using:

```
filter_var($rawTickets, FILTER_VALIDATE_INT, ['options' =>
['min_range' => 1, 'max_range' => 5]])
```

  4. Validate donation using filter_var($rawDonation, FILTER_VALIDATE_FLOAT) and verify that the value is between 0.0 and 500.0.
  5. If valid, calculate total cost ($tickets * 45.00 + $donation) and display the order summary; otherwise redisplay the sticky form with field-specific error messages.
- **Expected Output / Behavior:** Entering "invalid-email", 8 tickets, or -10 donation triggers specific validation errors. Entering user@example.com, 2 tickets, and 15.50 donation outputs Total Due: $105.50.
- **Constraints:** Must use filter_var() or filter_input() and check for === false so a 0.00 donation is treated as valid.
- **Self-Check & Grading Criteria:**
  - [ ] FILTER_VALIDATE_EMAIL used accurately (1 pt)
  - [ ] FILTER_VALIDATE_INT with min_range and max_range options array (2 pts)
  - [ ] FILTER_VALIDATE_FLOAT range check (=== false handled) and sticky form behavior (2 pts)

Question 13: Password Rules & Checkbox Array Validation (5 pts)

- **Learning Objective:** Validate password strength rules, process multi-checkbox array inputs (name="topics[]"), verify selections against an allowed list, and preserve sticky checkbox states.
- **Scenario:** A developer workshop registration form requires a password (at least 8 characters, containing at least one digit and one uppercase letter), a single mandatory "Terms of Service" checkbox, and between 1 and 3 workshop track checkboxes.
- **Starter File (q13_account_setup.php)**

- **Requirements:**
  1. Validate password: length at least 8 characters, contains an uppercase letter (preg_match('/[A-Z]/', $password)), and contains a digit (preg_match('/[0-9]/', $password)).
  2. Validate topics[]: read $_POST['topics'] ?? [] (verify is_array()), confirm every selected key exists in array_keys($availableTracks), and require at least 1 and at most 3 tracks.
  3. Validate agree_terms: ensure isset($_POST['agree_terms']).
  4. Make the workshop track checkboxes and the terms checkbox **sticky** on validation failure (checked attribute). Never repopulate the password input.
- **Expected Output / Behavior:** If the password fails validation while two tracks are checked, the form reloads showing the error message, an empty password field, and the two selected track checkboxes still checked.
- **Constraints:** Password input must NOT be sticky; checkboxes MUST remain sticky.
- **Self-Check & Grading Criteria:**
  - [ ] Password length, uppercase, and digit validation (2 pts)
  - [ ] Multi-checkbox topics[] array validation and whitelist check (2 pts)
  - [ ] Sticky checkboxes and cleared password box on error (1 pt)

Question 14: XSS Defense & Redirect-After-Success Pattern (5 pts)

- **Learning Objective:** Prevent Cross-Site Scripting (XSS) using htmlspecialchars() and implement the Run / Redisplay / Redirect decision pattern using a query string + header('Location: ...') + exit.
- **Scenario:** A support ticket submission page allows users to preview their message or submit it immediately. If the user clicks **"Preview"**, the page redisplays with an XSS-escaped preview box. If the user clicks **"Submit Final"** and inputs are valid, the script redirects via an HTTP Location header with a query string to q14_thankyou.php.
- **Starter Files:** Create q14_guestbook_preview.php and q14_thankyou.php.
- **Requirements:**
  1. In q14_guestbook_preview.php, provide form inputs for username (3–30 chars), message (5–200 chars), and two submit buttons:

```
<button type="submit" name="action" value="preview">Preview</button>
<button type="submit" name="action" value="submit">Submit Final</button>
```

  2. Validate inputs on `POST`. If invalid, **redisplay** the form with errors.
  3. If valid and `action === 'preview'`, **redisplay** the form along with a preview box rendering the escaped `$username` and `$message`.
  4. If valid and `action === 'submit'`, **redirect** before any HTML output using:

```
header('Location: q14_thankyou.php?user=' . urlencode($username));
exit;
```

  5. In q14_thankyou.php, read $_GET['user'] ?? 'Guest' and display a thank-you message escaped with htmlspecialchars().
- **Expected Output / Behavior:** Entering an HTML script payload such as <script>alert(1)</script> and clicking **Preview** prints the literal characters harmlessly on the page without executing JavaScript. Clicking **Submit Final** redirects the browser URL to q14_thankyou.php?user=....

- **Constraints:** No sessions allowed; pass state strictly via the URL query string. header('Location: ...') must be called before any HTML is sent, followed immediately by exit;.
- **Self-Check & Grading Criteria:**
  - [ ] Zero HTML/whitespace output before header('Location: ...') and exit (2 pts)
  - [ ] Proper use of urlencode() for query string building and htmlspecialchars(..., ENT_QUOTES, 'UTF-8') for HTML output (2 pts)
  - [ ] Distinct handling for Preview (redisplay) vs. Submit (redirect) (1 pt)

Part 3: Core Tasks — Introductory SQL (Questions 15–18)

*(Write and test all SQL statements for Questions 15–18 in **phpMyAdmin** on your local XAMPP server using the webprog2_midterm database, and save each script as a .sql file in your lab folder.)*

Question 15: SQL Table Design, Keys, and Data Types (q15_books_schema.sql) (5 pts)

- **Learning Objective:** Write a SQL CREATE TABLE statement selecting appropriate SQL data types (INT, VARCHAR, DECIMAL, DATE, TINYINT), column constraints (NOT NULL, DEFAULT), and an auto-incrementing PRIMARY KEY, then verify the table with SELECT.
- **Scenario:** A campus bookstore needs a books table in the webprog2_midterm database to store book metadata, exact retail price, publication date, and availability status.
- **Starter File:** Create q15_books_schema.sql.
- **Requirements:**

  1. Write a CREATE TABLE books statement with the following columns:
     - book_id: integer, auto-incrementing PRIMARY KEY
     - isbn: variable-length string up to 17 characters, NOT NULL
     - title: variable-length string up to 120 characters, NOT NULL
     - retail_price: exact fixed-point decimal up to 6 total digits with 2 decimal places (DECIMAL(6,2)), NOT NULL
     - published_date: date (DATE), NOT NULL
     - in_print: boolean-style tiny integer (TINYINT(1)), NOT NULL DEFAULT 1
  2. After creating the table in phpMyAdmin, run SHOW COLUMNS FROM books; to verify the structure.

- **Expected Output / Behavior:** Running q15_books_schema.sql in phpMyAdmin creates the books table with book_id marked as PRI (auto_increment), retail_price as decimal(6,2), and in_print defaulting to 1.
- **Constraints:** Do not use FLOAT for currency (DECIMAL is required).
- **Self-Check & Grading Criteria:**
  - [ ] Appropriate SQL data types (INT, VARCHAR, DECIMAL(6,2), DATE, TINYINT(1)) (3 pts)
  - [ ] PRIMARY KEY, AUTO_INCREMENT, NOT NULL, and DEFAULT 1 defined accurately (2 pts)

Question 16: Basic SQL SELECT, WHERE, and ORDER BY (q16_catalog_queries.sql) (5 pts)

- **Learning Objective:** Write SQL SELECT queries using WHERE filtering (BETWEEN, IN, LIKE, comparison operators) and ORDER BY sorting.
- **Scenario:** The store manager needs three read-only SQL reports extracted from the products table in webprog2_midterm.
- **Starter File:** Create q16_catalog_queries.sql.
- **Requirements:**
    1. **Query A (Featured Mid-Range Items):** Select sku, product_name, category, and unit_price from products where is_featured = 1 AND unit_price is between 50.00 and 350.00, ordered by unit_price DESC.
    2. **Query B (Storage & Audio Restock Watch):** Select sku, product_name, category, and stock_qty from products where category IN ('Storage', 'Audio') AND stock_qty is less than 25, ordered by stock_qty ASC.
    3. **Query C (Keyword Search):** Select sku, product_name, and unit_price from products where product_name LIKE '%SSD%', ordered by product_name ASC.
- **Expected Output / Behavior (in phpMyAdmin):**
    - Query A returns 4 rows: HW-103 ($329.00), HW-110 ($115.00), HW-101 ($89.99), and HW-105 ($79.99).
    - Query B returns 3 rows: HW-110 (9), HW-112 (15), and HW-106 (18).
    - Query C returns 2 rows: 1TB NVMe M.2 SSD and 2TB Portable External SSD.
- **Constraints:** Select only the requested columns (do not use SELECT *).
- **Self-Check & Grading Criteria:**
    - [ ] Query A uses WHERE is_featured = 1 AND unit_price BETWEEN 50.00 AND 350.00 ORDER BY unit_price DESC (2 pts)
    - [ ] Query B uses WHERE category IN ('Storage', 'Audio') AND stock_qty < 25 ORDER BY stock_qty ASC (2 pts)
    - [ ] Query C uses WHERE product_name LIKE '%SSD%' ORDER BY product_name ASC (1 pt)

Question 17: SQL Pagination Queries with LIMIT and OFFSET (q17_pagination_queries.sql) (5 pts)

- **Learning Objective:** Use SQL ORDER BY, LIMIT, and OFFSET clauses to retrieve specific pages of rows and top-N rankings.
- **Scenario:** The products table contains 12 items. You need to write the exact SQL queries that power a 4-items-per-page catalog view (Page 1, Page 2, Page 3) as well as a "Top 3 Cheapest Accessories" widget.
- **Starter File:** Create q17_pagination_queries.sql.
- **Requirements:**
    1. **Query A (Page 1 — Rows 1 to 4):** Select product_id, sku, product_name, and unit_price from products ordered by product_id ASC with a limit of 4 and offset of 0.
    2. **Query B (Page 2 — Rows 5 to 8):** Same query for Page 2 (LIMIT 4 OFFSET 4).
    3. **Query C (Page 3 — Rows 9 to 12):** Same query for Page 3 (LIMIT 4 OFFSET 8).
    4. **Query D (Top 3 Cheapest Accessories):** Select sku, product_name, and unit_price from products where category = 'Accessories', ordered by unit_price ASC, limited to 3 rows.
- **Expected Output / Behavior (in phpMyAdmin):**

  - Query A returns HW-101 through HW-104.
  - Query B returns HW-105 through HW-108.
  - Query C returns HW-109 through HW-112.
  - Query D returns HW-109 ($14.99), HW-111 ($29.99), and HW-104 ($34.95).
- **Constraints:** Always include ORDER BY before LIMIT and OFFSET so row order is deterministic.
- **Self-Check & Grading Criteria:**
  - [ ] Correct LIMIT 4 OFFSET 0, OFFSET 4, and OFFSET 8 pagination queries (3 pts)
  - [ ] Correct WHERE + ORDER BY unit_price ASC + LIMIT 3 in Query D (2 pts)

Question 18: SQL Aggregate Functions (q18_aggregate_queries.sql) (5 pts)

- **Learning Objective:** Write SQL queries using aggregate functions (COUNT, SUM, AVG, MIN, MAX) with column aliases (AS) and WHERE filtering.
- **Scenario:** Management needs summary statistics calculated directly in SQL from the products and courses tables.
- **Starter File:** Create q18_aggregate_queries.sql.
- **Requirements:**

1. **Query A (Overall Hardware Inventory KPI):** Write a single SELECT query on products returning:
   - COUNT(*) AS total_products
   - SUM(stock_qty) AS total_units
   - AVG(unit_price) AS avg_price
   - MIN(unit_price) AS min_price
   - MAX(unit_price) AS max_price
   - SUM(unit_price * stock_qty) AS total_inventory_value
2. **Query B (Online Courses Summary):** Write a SELECT query on courses with WHERE is_online = 1 returning:
   - COUNT(*) AS online_course_count
   - AVG(lab_fee) AS avg_online_lab_fee
   - SUM(max_seats) AS total_online_seats

- **Expected Output / Behavior (in phpMyAdmin):**
  - Query A returns total_products: 12, total_units: 468, avg_price: 128.075000, min_price: 14.99, max_price: 499.99, total_inventory_value: 27482.55.
  - Query B returns online_course_count: 3, avg_online_lab_fee: 43.333333, total_online_seats: 98.
- **Constraints:** Must use exact column aliases specified above.
- **Self-Check & Grading Criteria:**
  - [ ] All 6 aggregate expressions and aliases in Query A (3 pts)
  - [ ] WHERE is_online = 1 combined with COUNT, AVG, and SUM in Query B (2 pts)

Part 4: Challenge & Integrated Mini-App Tasks (Questions 19–22)

Question 19 (Challenge): Server Access Log Analyzer & Filter Engine (10 pts)

- **Learning Objective:** Combine declare(strict_types=1), modular functions.php, multidimensional arrays, match expressions, $_GET allowed-list and range validation, and XSS-safe reporting.
- **Scenario:** A DevOps dashboard analyzes an array of server request logs. Administrators can filter logs by HTTP method (GET, POST, ALL) and minimum response time in milliseconds (min_ms between 0 and 5000) via a GET filter form.
- **Starter Files & Data:** Create q19_functions.php and q19_log_analyzer.php. Use the dataset inside q19_log_analyzer.php
- **Requirements:**
    1. In q19_functions.php (with declare(strict_types=1);):
        - Define getStatusBadge(int $status): string using a match (true) expression:
            - Status 500 or above returns 'CRITICAL (5xx)'
            - Status 400 or above returns 'WARNING (4xx)'
            - Status 200 or above returns 'OK (2xx)'
            - default returns 'INFO'
        - Define filterLogs(array $logs, string $method = 'ALL', int $minMs = 0): array that iterates through $logs and returns only entries matching $method (unless 'ALL') and having 'ms' greater than or equal to $minMs.
    2. In q19_log_analyzer.php, validate $_GET['method'] ?? 'ALL' against ['ALL', 'GET', 'POST'] and $_GET['min_ms'] ?? 0 using filter_var (0 to 5000).
    3. Call filterLogs() using **named arguments**, compute the average response time of the filtered rows, and render a sticky GET form and HTML table.
- **Expected Output / Behavior:** Filtering by method=POST and min_ms=100 returns 2 rows (/login.php with WARNING (4xx) and /checkout.php with CRITICAL (5xx)) and displays Matching Entries: 2 | Average Response Time: 800.0 ms.
- **Constraints:** Must use declare(strict_types=1), match, named arguments, and strict validation.
- **Self-Check & Grading Criteria:**
    - [ ] q19_functions.php uses declare(strict_types=1) and match expression accurately (3 pts)
    - [ ] filterLogs() called with named arguments and filters accurately (3 pts)
    - [ ] Sticky GET form validates method whitelist and min_ms integer range (2 pts)
    - [ ] Average response time calculation and escaped table output (2 pts)

Question 20 (Integrated Mini-App 1): University Course Finder & Tuition Estimator (10 pts)

- **Learning Objective:** Build an integrated PHP mini-application combining modular includes (q20_functions.php), a sticky $_GET search/filter form, input validation, custom tuition calculation functions, and companion SQL SELECT queries (q20_course_queries.sql).
- **Scenario:** Students browsing the university catalog need to filter courses by department (ALL, CS, IT, Design), delivery mode (only_online checkbox), and keyword search (title), while seeing the estimated total cost per course (credits * $120.00 + lab_fee).
- **Starter Files & Data:** Create q20_functions.php, q20_course_finder.php, and q20_course_queries.sql. Use the array inside q20_course_finder.php

- **Requirements:**
    1. In q20_functions.php (declare(strict_types=1);):
        - Define calculateCourseCost(int $credits, float $labFee, float $creditRate = 120.00): float returning ($credits * $creditRate) + $labFee.
        - Define getDepartmentLabel(string $deptCode): string using match ($deptCode) ('CS' => 'Computer Science', 'IT' => 'Information Technology', 'Design' => 'Digital Design', default => 'General Studies').
    2. In q20_course_finder.php, validate $_GET inputs:
        - dept: allowed list ['ALL', 'CS', 'IT', 'Design'] (default 'ALL').
        - keyword: trimmed string, max 40 characters (if longer than 40 characters, record a validation error message and reset keyword to '').
        - only_online: checkbox (isset($_GET['only_online'])).
    3. Filter $courses in PHP using foreach (matching department, case-insensitive keyword via stripos(), and online status) and render the sticky filter form and an HTML table showing code, title, full department label, credits, fee, format (Online or In-Person), and estimated tuition.
    4. In q20_course_queries.sql, write two SQL SELECT queries against the courses table in phpMyAdmin:
        - **Query 1:** Select course_code, title, credits, lab_fee, and (credits * 120 + lab_fee) AS est_tuition from courses where department = 'CS' ordered by lab_fee ASC.
        - **Query 2:** Select course_code, title, department, and max_seats from courses where is_online = 1 AND title LIKE '%Design%' ordered by course_code ASC.
- **Expected Output / Behavior:** Filtering by Department CS displays the 4 CS courses with calculated tuition totals ($530.00, $555.00, $420.00, $570.00). Both SQL queries run cleanly in phpMyAdmin.
- **Constraints:** Must validate all $_GET inputs and escape all HTML output with htmlspecialchars().
- **Self-Check & Grading Criteria:**
    - [ ] Strict-typed q20_functions.php with calculateCourseCost() and match-based getDepartmentLabel() (3 pts)
    - [ ] Whitelist validation on dept, length validation on keyword, checkbox handling for only_online (3 pts)
    - [ ] Sticky form controls and XSS-escaped table rendering (2 pts)

- [ ] Both companion SQL SELECT queries in q20_course_queries.sql execute accurately (2 pts)

Question 21 (Integrated Mini-App 2): Hardware B2B Quote Builder with Redirect-After-Success (10 pts)

- **Learning Objective:** Integrate multi-checkbox arrays, $_POST validation, filter_var integer validation, PHP 8 match promo codes, sticky form state on error, the **redirect-after-success** pattern (header('Location: ...') + query string + exit), and companion SQL queries.
- **Scenario:** A corporate buyer uses a quote builder page (q21_quote_builder.php) to enter their company name, select one or more hardware products via checkboxes, specify a uniform bundle quantity per item (1 to 20), and optionally enter a promo code (SAVE10 or VIP20). If validation fails, the form **redisplays** with sticky inputs and errors. If validation succeeds, the script calculates the quote totals and **redirects** via query string (header('Location: q21_quote_success.php?...') + exit) to a confirmation receipt page.
- **Starter Files & Data:** Create q21_quote_builder.php, q21_quote_success.php, and q21_quote_queries.sql. Use the catalog array in q21_quote_builder.php
- **Requirements:**
  1. In q21_quote_builder.php, validate POST submissions:
     - company_name: trimmed length between 2 and 60 characters.
     - product_ids: array of selected checkboxes ($_POST['product_ids'] ?? []), at least 1 valid catalog key selected, each validated as an integer present in $catalog.
     - bundle_qty: integer between 1 and 20 using filter_var().
     - promo_code: uppercase trimmed string matched via match ($promoCode) ('' => 0.0, 'SAVE10' => 0.10, 'VIP20' => 0.20, default => -1.0 which adds an "Invalid promo code" error).
  2. If validation fails, **redisplay** the form with error messages and sticky values (company name, selected checkboxes, quantity, promo code).
  3. If validation succeeds, calculate $subtotal, $discountAmount, and $finalTotal, build a query string using http_build_query(), and **redirect** using header('Location: q21_quote_success.php?' . $queryString); exit;.
  4. In q21_quote_success.php, read the query string parameters from $_GET, escape all values with htmlspecialchars(), and display the formatted quote confirmation.
  5. In q21_quote_queries.sql, write two SQL queries against the products table:
     - **Query 1:** Select product_id, sku, product_name, unit_price, and stock_qty from products where sku IN ('HW-101', 'HW-103', 'HW-107') ordered by product_name ASC.
     - **Query 2:** Select sku, product_name, and stock_qty from products where stock_qty < 15 ordered by stock_qty ASC.
- **Expected Output / Behavior:** Submitting invalid data redisplays sticky form controls; submitting valid data redirects the browser to q21_quote_success.php?company=...&items=...&qty=...&total=... and displays the final quote.
- **Constraints:** No sessions allowed. Must use header('Location: ...') + exit with a query string on success.
- **Self-Check & Grading Criteria:**
  - [ ] Validation of company name, checkbox whitelist, bundle quantity range (1–20), and promo code via match (3 pts)
  - [ ] Sticky checkboxes and inputs on validation failure (2 pts)
  - [ ] Redirect-after-success using http_build_query() / urlencode(), header('Location: ...'), and exit (3 pts)

- [ ] Escaped output on q21_quote_success.php and accurate SQL in q21_quote_queries.sql (2 pts)

Question 22 (Integrated Mini-App 3): Storefront Explorer & Quick-Order Redirect App (10 pts)

- **Learning Objective:** Combine declare(strict_types=1), custom helper functions (q22_functions.php), $_GET category/price filtering and array pagination, a $_POST quick-order form with redirect-after-success, and companion SQL aggregate and LIMIT/OFFSET queries (q22_explorer_queries.sql).
- **Scenario:** Build a storefront explorer (q22_store_explorer.php) that lets users filter products by category and maximum price via $_GET, computes live summary metrics (count, average price, min price, max price) for the filtered items, and allows placing a quick reservation via $_POST that redirects on success to q22_order_confirmed.php.
- **Starter Files & Data:** Create q22_functions.php, q22_store_explorer.php, q22_order_confirmed.php, and q22_explorer_queries.sql. Use the dataset in q22_store_explorer.php:
- **Requirements:**

1. In q22_functions.php (declare(strict_types=1);):
    - Define getStockBadge(int $qty): string using match (true) ($qty <= 0 => 'Out of Stock', $qty < 15 => 'Low Stock', default => 'In Stock').
    - Define calculateCatalogStats(array $items): array that returns ['count' => ..., 'avg' => ..., 'min' => ..., 'max' => ...] for a filtered product list.
2. In q22_store_explorer.php, validate $_GET filters:
    - category: allowed list ['ALL', 'Peripherals', 'Monitors', 'Accessories', 'Storage', 'Audio'].
    - max_price: filter_var float between 10.00 and 1000.00 (default 500.00).
    - Filter $products, compute summary stats via calculateCatalogStats(), and display both the metrics banner and the filtered product table.
3. On the same page, include a **Quick Reserve (POST)** form with buyer_email (validated via FILTER_VALIDATE_EMAIL), sku (validated against allowed SKUs in $products), and qty (integer 1 to 10 via FILTER_VALIDATE_INT).
    - If POST validation fails, **redisplay** the page with error messages and sticky inputs.
    - If POST validation succeeds, **redirect** using header('Location: q22_order_confirmed.php?' . http_build_query([...])); exit;.
4. In q22_explorer_queries.sql, write the two equivalent SQL queries on the products table:
    - **Query 1 (Filtered Aggregates):** Select COUNT(*) AS total_matches, AVG(unit_price) AS avg_price, MIN(unit_price) AS min_price, and MAX(unit_price) AS max_price from products where unit_price <= 100.00.

    - **Query 2 (Filtered Pagination):** Select sku, product_name, category, unit_price, and stock_qty from products where unit_price <= 100.00 ordered by unit_price ASC with LIMIT 3 OFFSET 0.

- **Expected Output / Behavior:** Filtering by max_price=100 displays the matching items and summary statistics; submitting a valid Quick Reserve redirects to q22_order_confirmed.php showing the escaped confirmation details.
- **Constraints:** No sessions or database driver patterns in PHP; use header('Location: ...') + exit with query parameters for the redirect.
- **Self-Check & Grading Criteria:**
    - [ ] q22_functions.php implements getStockBadge() (match) and calculateCatalogStats() (3 pts)
    - [ ] $_GET filtering and $_POST validation (FILTER_VALIDATE_EMAIL, SKU whitelist, FILTER_VALIDATE_INT) (3 pts)
    - [ ] Redirect-after-success to q22_order_confirmed.php using query string + header('Location: ...') + exit (2 pts)
    - [ ] Companion SQL aggregate and LIMIT/OFFSET queries in q22_explorer_queries.sql (2 pts)